Playing Card Image Classification

Alexander Ottelien, Daniel Fischer, Wyatt Jens

CSC 2611-121

Professor Gabriel Wright

8 December 2023

Abstract

In this project, we took on the challenging task of classifying playing card images, working with a diverse dataset that included thousands of card images. Our approach was centered around a Convolutional Neural Network (CNNs) and image preprocessing. Through tedious training and fine-tuning, our model achieved commendable accuracy in predicting playing cards, showcasing adaptability to real-world applications. Our model achieved a training accuracy of 84.84% and a validation accuracy of 83.40%.

Introduction

For our project, we wanted to be able to accurately recognize and predict images of playing cards-ace through king for spades, clubs, hearts, and diamonds, including the jokers, totaling 53 different card classes. Our goal was to reach above an 80% accuracy rating for our model, as we had never practiced predicting so many different classes before. Initially, we wanted to use the model to predict poker hands and their winning probabilities. However, due to time constraints, we shifted focus to the primary task. We chose the Cards Images Dataset-Classification from Kaggle[1] to train our model, which provided ample data with over 7600 images. Each card class featured a minimum of 100 unique images. The dataset's variety, including standard and unconventional card designs, provided a robust foundation for our model.

Baseline Model

Our baseline model is a CNN designed for image classification. It consists of several layers to process and learn features from input images. The input shape is set to (150, 150, 3), indicating images of size 150x150 pixels with three color channels (RGB). The model begins with a convolutional layer with 32 filters and a kernel size of (3, 3), followed by another convolutional layer with 64 filters. Each convolutional layer is activated using Rectified Linear Unit (ReLU) activation function to introduce non-linearity.

After both convolutional layers, a max-pooling layer with a pool size of (2, 2) is applied to down sample the spatial dimensions of the feature maps. The Flatten layer is then used to flatten the 2D feature maps into a 1D vector. Two densely connected (fully connected) layers are incorporated for further feature extraction. The first layer consists of 128 neurons with ReLU activation, while the final layer consists of num_class neurons (53 in this case) and employs a SoftMax activation function, making it suitable for multi-class classification.

The model is compiled using sparse categorical cross entropy as the loss function, stochastic gradient descent (SGD) as the optimizer, and accuracy as the evaluation metric. The training is then performed for 25 epochs with a batch size of 32. This model achieved a validation accuracy of about 73% and a training accuracy of about 99%. This indicates a significant issue with overfitting in the model. Additionally, it's worth noting that beyond epoch 10, the accuracy for both validation and training plateaus, remaining unchanged.

Model Improvement Approaches

The first problem that we wanted to address with our model was its overfitting. Our baseline model had achieved a validation accuracy of about 73% while its training accuracy was at about 99%. We decided that the best way to address overfitting was to add some dropout layers within our model. In total, we added two different dropout layers with varying dropout values. We added the first dropout layer directly before our flatten layer and gave it a dropout value of .25. The second dropout layer we added was added just before the last dense layer that we had, and it had a dropout value of .3. We chose these specific locations for the dropout layers as we thought that they would have the most effect on the overfitting. The first one was to drop the nodes it used right before switching from 2D to 1D vectors, and the second was to clean it up right before the final decision for its class was made.

The second problem that we had to address was the fact that we were below our target accuracy. As stated, we had managed an accuracy of around 70% but we wanted an accuracy of above 80%. We decided thar we were going to add three more convolutional layers, each with a max-pooling layer with a pooling size of (2, 2) after it, to achieve our target accuracy. All these layers were added in the same place, before the first dropout layer. The first of the new convolutional layers had a neuron count of 32, the second layer had a neuron count of 128, and the final layer had a neuron count of 64.

After adding the dropout and convolutional layers, our model had achieved a validation accuracy of about 83% while the training accuracy was about 84%. Using this new model, we achieved our goal of an 80% accuracy at predicting 53 different types of cards, while having nearly no overfitting.

Results/Model Comparison

Throughout this project, three main models were compared in order to determine the most effective one. The first model was the baseline model as described before, being a CNN model including 6 layers of various convolutional layers, max pooling layers, a flatten layer, and dense layers. This model achieved a validation accuracy of 73%, but with a training accuracy of 99%. These results showed severe overfitting with just mediocre accuracy. Because of these results, improvements were made to the model. This included the addition of dropout layers, more convolutional layers, and more total layers. As stated before, this helped with the overfitting issues as well as the mediocre accuracy issues. This new optimized CNN model achieved a validation accuracy of 83.40% while having a training accuracy of right around 84.84%. To test our model effectively we needed to compare it to a different type of model to evaluate the strengths and weaknesses of our optimized model.

The new type of model to be used as a comparison to our convolutional neural network was a deep neural network (DNN) as traditional machine learning models like kNNs or k-Means models are not particularly suited for image classification. The result of this model that consisted of mainly dense layers was a validation accuracy of 14% and a training accuracy of about 11% (when ran with the same hyperparameters as the improved CNN model). This shows an extremely lower accuracy in both the validation and training sets when compared with our best model results. While the accuracy of the DNN was a lot lower, it is important to note the improved runtime the DNN had over the CNN. The DNN ran around 5 seconds per epoch while the improved CNN model ran at around 25 seconds per epoch. This was not surprising as the DNN model had significantly less layers than the CNN.

The improved CNN model was by far the best model we achieved throughout the project, which wasn't a surprising result. From a base model standpoint, CNN models are known for their prowess in image classification. This is the main reason we chose this type of model for this problem in the first place. The convolutional layers containing the kernels are key to the model's success at classifying images. This is most likely the reason that the DNN model could not come close to matching the CNN model's accuracies.

Conclusion

The objective of this project was to use a model to accurately predict the type and suit of a card given an image of that card. This presented a challenge as there are 53 different kinds of playing cards (including jokers) that we wanted our model to be able to accurately predict. Through our experiments with both convolutional and deep neural networks, distinct patterns and trends emerged. The first being that CNN models outclasses DNN models in terms of their accuracy of image classification by a large magnitude. A second is the prevalence and consistent issue overfitting can be. One of the most consistent modifications needing to be made to the model throughout the project was combating overfitting of the dataset. This brings in the necessity of hyperparameter tuning as well as model architecture modification to ensure as good of a model as possible.

While this dataset was large and diverse, there weren't many limitations encountered in terms of how good the data was, or the amount of data used. There were some cards within the dataset that were pretty outrageous in terms of design and color, sometimes only the corner numbers and suit shape were visible within the entire card. While these cards may have made training slightly harder, there did not seem to be any significant limitations that occurred because of these kinds of cards, mainly because these images did represent actual playing cards and helped to strengthen our model against all kinds of real-life application. With additional time and effort invested in optimizing and testing, the model could have a wide range of applications, presenting numerous opportunities for further development. This model could be used to classify what hand of cards a player may have in games of poker as the model could be used to classify each card of the hand and use this to state what hand a player has or the strength of this hand. This, of course, would have to be with an even more optimized model that can predict the type and suit of a card even more accurately than this model.

Through this project of creating a model to predict the type and suit of a playing card by its image, an efficient and accurate model was developed that has the potential in any number of future real-world applications.

References

[1] Gerry. “Cards Image Dataset-Classification.” *Kaggle*, 17 Nov. 2022, www.kaggle.com/datasets/gpiosenka/cards-image-datasetclassification.